RE:봄 주말 교육 · 1부 · 2026-11-07(토)

LLM과 API 기초

API를 부르는 법부터 토큰 · 비용 · 파라미터까지

실습 노트북 01 API 기초 · 02 토크나이저 · 12 Responses API (코랩)

오리엔테이션

9시간 지도: 기초를 먼저 쌓고, 프로젝트는 질문으로 잇는다

	날짜	주제	다루는 것
1부	11/07	LLM과 API 기초	API · 토큰 · LLM 동작 원리 · 추론 모델 · 비용과 캐싱 · 파라미터
2부	11/14	모델 다루기와 임베딩	오픈 모델 · 양자화 · 로컬 서빙 · 프롬프트 설계 · 임베딩과 유사도 검색
3부	11/21	검색 · RAG · 에이전트	청킹 · RAG · 하이브리드 검색 · 툴 콜링 · 에이전트
W4	11/28	적용 클리닉	새 자료 없음. 각자 담당 코드를 가져와 같이 본다

RE:봄 블록마다 **RE:봄이라면?** 질문이 한 장씩 있다. 답은 화요일 리뷰에서 같이 맞춰 본다

오리엔테이션

오늘의 흐름: 설명하고 바로 실습하기를 세 번 반복한다



시작	블록	설명 + 심화	실습
0:00	오리엔테이션	지도 · 실습 환경 · 키 규칙	
0:10	A · API 기초	요청과 응답 · 상태 코드 · SDK · FastAPI	01 API 기초
1:15	B · 토큰과 LLM의 동작	토큰 · 컨텍스트 · 다음 토큰 예측 · 정렬 · 환각	02 토크나이저
2:55	C · API를 제대로 쓰기	추론 · 평가 · 비용 · 캐싱 · 파라미터 · 응답 객체	12 Responses API
4:20	정리	열 문장 요약 · 화요일까지 할 일	

심화 있다 보라색 **심화** 장은 본 장을 한 단계 더 판 내용이다. 실습 뒤에는 결과 해석 장이 한 장씩

오리엔테이션

실습 환경과 API 키 규칙

실습 환경

- 코랩에서 github.com/acho98/rebom-lec 의 labs/ 노트북을 연다
- 01 · 02 · 12는 CPU 런타임이면 된다
- 데이터는 노트북이 저장소에서 알아서 받는다
- 노트북 맨 위의 코랩 배지를 누르면 바로 열린다

API 키 규칙

- 키는 노트북의 입력창(getpass)에만 붙여 넣는다
- 코드 · 출력 · 깃 · 메신저에 키를 쓰지 않는다
- 수업용 키는 지출 한도가 걸려 있고, 수업 뒤 폐기한다
- 공개 저장소에 올라간 키는 발견 즉시 자동 폐기된다

주의 키가 새면 순서는 **폐기가 먼저**, 이력 정리는 나중이다. 지워도 이미 복제됐을 수 있다

A

API 기초

프로그램끼리는 어떻게 말을 주고받을까?

- 요청과 응답 · 상태 코드
- requests vs SDK · OpenAI 호환 규격
- AI API의 네 가지 종류 · FastAPI로 내 API 만들기

설명 20분 · 심화 20분 → 실습 01 · 20분 → 결과 해석

API는 요청과 응답의 약속이다

정해진 주소로 **요청**을 보내면, 정해진 형식으로 **응답**이 돌아온다

요청: 내 코드 → 서버

```
POST /v1/responses
Host: api.openai.com
Authorization: Bearer <키>
Content-Type: application/json

{"model": "gpt-6-luna",
 "input": "안녕?"}
```

응답: 서버 → 내 코드

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "completed",
 "output": [{
   "type": "message",
   "content": [{
     "text": "안녕하세요!"}]}]}
```

핵심 웹페이지도 같은 HTTP다. 다른 점은 받는 쪽이 사람이 아니라 **프로그램**이라는 것

요청은 네 부분: 메서드 · 주소 · 헤더 · 본문

부분	예	뜻
메서드	GET · POST	가져오기 · 보내서 처리하기
주소 (URL)	https://api.openai.com/v1/responses	어느 서버의 어느 기능인가
헤더	Authorization: Bearer 〈키〉	누가 보내나 (인증) · 본문 형식
본문	{"model": "gpt-6-luna", "input": "안녕?"}	무엇을 처리할까 (JSON)

핵심 GET은 조건을 주소 뒤에 붙인다: ?page=1&size=100. 공공데이터 API가 대부분 이 방식이다

A · API 기초 · 심화

POST는 다시 보내면 두 번 처리될 수 있다: 멱등성

메서드	멱등	같은 요청을 다시 보내면
GET	예	같은 데이터를 한 번 더 받을 뿐이다
PUT · DELETE	예	결과가 같다 (같은 값으로 덮기 · 이미 지운 것)
POST	아니오	**두 번 처리**될 수 있다 (결제가 두 번)

```
POST /api/payments HTTP/1.1
Content-Type: application/json
Idempotency-Key: 8f7a2c91-4e21-4d78-91f1-a0b2c3d4e5f6

{"orderId": "ORDER-1001", "amount": 10000,
 "currency": "KRW"}
```

Idempotency-Key

- 서버는 처리했는데 **응답만 유실**될 수 있다
- 같은 키로 다시 오면 첫 결과를 돌려준다

심화 LLM 호출도 POST다. 응답만 잃고 재시도하면 **두 번 과금**된다. 429로 거절된 요청은 과금되지 않는다

심화

공공데이터포털 키는 두 개: params에는 Decoding 키를 넣는다

키	생김새	넣는 곳
Encoding 키	이미 인코딩됨 (%2B · %3D)	URL 문자열에 직접 붙일 때
Decoding 키	원문 (+ · / · =)	requests의 params

```
requests.get(URL, params={"serviceKey": ENCODING_KEY})
# params가 한 번 더 인코딩: % → %25  →  SERVICE_KEY_IS_NOT_REGISTERED_ERROR
```

오류인데 HTTP 200이 온다

- 본문의 resultCode(게이트웨이 오류는 returnReasonCode)로 알린다. 상태 코드만 보면 성공으로 착각한다
- 기본 응답이 XML인 서비스가 많다. _type=json을 줘도 오류는 XML로 올 수 있어 **JSON 파싱 실패도 오류**로 잡는다

심화

주소에 키가 들어가는 API는 로그에 남기기 전에 키를 가린다

raise_for_status() · 연결 오류의 예외 메시지와 서버 · 프록시 로그에 **주소 전체**가 남는다. str(e)를 그대로 찍지 않는다

```
from urllib.parse import urlsplit, urlunsplit, parse_qsl, urlencode
def mask_url(url):
    p = urlsplit(url)
    q = [(k, "***" if k.lower() == "servicekey" else v)
         for k, v in parse_qsl(p.query, keep_blank_values=True)]
    # safe="*"가 없으면 ***가 %2A%2A%2A로 찍힌다
    return urlunsplit(p._replace(query=urlencode(q, safe="*")))
try: r.raise_for_status()
except requests.HTTPError:
    log.error("API 실패 status=%s url=%s", r.status_code, mask_url(r.url))
# → API 실패 status=500 url=https://api.example.com/data?serviceKey=***&page=1
```

상태 코드 첫 자리로 누구 문제인지부터 가른다

2xx · 성공

- 200 OK
- 본문(JSON)을 읽는다

4xx · 내 요청 문제

- 400 형식 · 파라미터 오류
- 401 키가 없거나 틀림
- 403 권한 없음
- 404 주소 · 모델 이름 틀림
- 422 값 검증 실패
- 429 너무 많이 · 한도 초과

5xx · 서버 문제

- 500 · 502 · 503
- 제공자 장애 · 과부하
- 잠시 뒤 다시 보낸다

핵심 4xx는 **고쳐서** 다시 보낸다. 429와 5xx만 **기다렸다가** 다시 보낸다

심화

429: 대기 시간은 헤더로, 재시도 여부는 error.code로

헤더 (OpenAI)	뜻
Retry-After	몇 초 뒤에 다시 보내라. 있으면 이 값을 따른다
x-ratelimit-remaining-requests · -tokens	남은 요청 수 · 토큰 수
x-ratelimit-reset-requests · -tokens	다시 채워지기까지 남은 시간

rate_limit_exceeded

- 분 · 일 단위 속도 한도
- **기다리면 풀린다** → 백오프 후 재시도

insufficient_quota

- 결제 한도 · 잔액 소진
- **기다려도 안 풀린다** → 멈추고 사람에게 알림

심화 remaining 헤더는 정상 응답에도 붙어 온다. 줄어들면 **429를 맞기 전에** 속도를 늦춘다

지수 백오프에 지터를 섞어 재시도 시점을 흩뜨린다

재시도	1	2	3	4	5	6
지수 백오프	1초	2초	4초	8초	16초	30초
+ full jitter	0~1	0~2	0~4	0~8	0~16	0~30 무작위

```python
import random, time

base, cap = 1, 30
for attempt in range(6):
    wait = min(cap, base * 2 ** attempt)
    time.sleep(random.uniform(0, wait))
    # ...재시도
```

워커 100개가 동시에 429를 받으면

- 백오프만: 1 · 2 · 4초 뒤 **같은 순간**에 다시 몰려 또 429
- 지터: 재시도 시각이 흩어져 서버가 받을 수 있는 만큼씩 들어간다

심화 백오프는 간격을 늘리고 지터는 시점을 흩뜨린다. **요청량은 줄지 않는다**: 계속 넘치면 큐 · 동시성 제한이 필요하다

같은 호출, 두 가지 방법: requests로 직접 vs SDK

requests: HTTP를 직접 만든다

```python
import os, requests
key = os.environ["OPENAI_API_KEY"]
r = requests.post(
    "https://api.openai.com/v1/responses",
    headers={"Authorization": f"Bearer {key}"},
    json={"model": "gpt-6-luna", "input": "hi"},
)
print(r.status_code)          # 200
print(r.json()["output"])     # 답이 깊이 들어 있다
```

OpenAI SDK

```python
from openai import OpenAI
client = OpenAI()  # 키: 환경변수
r = client.responses.create(
    model="gpt-6-luna",
    input="hi",
)
print(r.output_text)
```

핵심 SDK가 대신 해 주는 것: 주소 · 인증 헤더 · JSON 변환 · 타입 · **재시도(기본 2회)**

심화

재시도는 층마다 곱해진다: 3 × 3 × 3 = 27번

층	설정	최대 시도
① Airflow 태스크	retries=2	3번
② 내 파이썬 코드	for attempt in range(3)	3번
③ OpenAI SDK	max_retries=2 (기본값)	3번

27번

작업 하나가 보낼 수 있는 최대 요청

서버가 처리했다면 그만큼 과금

상황	재시도를 맡길 층
단순 LLM 호출	SDK 기본 재시도
지터 · 재시도 정책을 직접 정해야 함	내 코드 (SDK는 max_retries=0)
작업 전체를 처음부터 다시 해야 함	Airflow 재시도 + 작업 멱등성
여러 에이전트가 같은 API를 부름	게이트웨이에서 재시도 · 한도를 한곳에서

심화

재시도는 한 층에서만, 타임아웃은 작업에 맞게 줄인다

```python
import openai, random, time
client = openai.OpenAI(max_retries=0, timeout=30)   # SDK 재시도 끔, 600초 → 30초
RETRY = (openai.RateLimitError, openai.APIConnectionError,
         openai.InternalServerError)                # 429 · 연결 오류 · 5xx
for attempt in range(3):
    try:
        r = client.responses.create(model="gpt-6-luna", input=text); break
    except RETRY as e:
        if attempt == 2 or getattr(e, "code", None) == "insufficient_quota": raise
        time.sleep(random.uniform(0, 2 ** attempt))  # 지터 백오프
```

어디서	이렇게
requests	timeout이 없으면 **무한 대기** → timeout=(3, 30). 기준은 평소 응답 p99의 2배쯤
Airflow 태스크	retries=0으로 재시도는 코드 한 곳에서. 작업째 다시 돌리면 upsert로 멱등하게

OpenAI 호환 규격: base_url만 바꾸면 다른 서버도 같은 코드로

```
cloud = OpenAI()                                          # OpenAI
local = OpenAI(base_url="http://localhost:11434/v1",
               api_key="ollama")                          # 내 GPU의 Ollama
clova = OpenAI(base_url="https://clovastudio.stream.ntruss.com/v1/openai",
               api_key=CLOVA_KEY)                         # 네이버 클라우드

# 부르는 코드는 같다. 바뀌는 것은 model 이름뿐
r = local.chat.completions.create(model=LOCAL_MODEL, messages=msgs)
```

핵심 Ollama · vLLM · 여러 클라우드가 같은 규격을 따른다. 모델을 바꿔도 **코드는 거의 그대로**. 2부 실습 05에서 직접 해 본다

AI API는 하는 일에 따라 네 가지로 나뉜다

생성
- 글 · 이미지 → 글
- 답변 · 요약 · 추출 · 분류 (오늘)

임베딩
- 글 · 이미지 → 숫자 벡터
- 의미가 가까운 것 찾기 (2부)

검색
- 질의 → 관련 문서 목록
- 벡터 DB · 키워드 · 웹 (3부)

결정
- 입력 → 정해진 선택지 + 점수
- 라우팅 · 채점 · 안전 판정

핵심 같은 모델이라도 **어떤 API로 부르느냐**에 따라 돌려받는 것이 다르다

내 함수도 API가 된다: FastAPI

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Req(BaseModel):
    text: str
    max_len: int = 50

@app.post("/summarize")
def summarize(req: Req):
    return {"summary": req.text[:req.max_len]}
```

FastAPI가 알아서 해 주는 것

- /docs 에 문서 화면이 생긴다
- JSON ↔ 파이썬 객체 변환
- 타입이 틀리면 422 + 어느 필드가 틀렸는지 알려 준다
- 실습 01 §4~5에서 직접 422를 받아 본다

배운 것을 우리 프로젝트에 대 보기

RE:봄에는 API가 세 방향으로 나온다: 받아 오는 API(공공데이터 구조동물) · 부르는 API(임베딩 · 생성) · 만드는 API(검색)

Q1 공공데이터 API가 429나 503을 돌려주면, 수집 작업은 어떻게 해야 할까?

Q2 우리가 만들 검색 API는 어떤 입력에 422를 돌려줘야 할까?

Q3 공공데이터 키와 OpenAI 키는 어디에 두고, 코드에서 어떻게 읽을까?

지금 답하지 않아도 된다. 화요일 리뷰에서 각자 담당에 가까운 질문 하나씩 같이 본다

Q1 수집: 호출 수부터 줄이고, 끊긴 곳부터 다시 받는다

numOfRows	10,000건에 필요한 호출
10	1,000번
100	100번
1,000	**10번**

오류별 대응

- 503 · 타임아웃: 지터 백오프로 3~5회
- 코드 22 (일일 한도 소진): **재시도하지 말고** 멈추고 알린다
- 429는 일일 한도가 아니라 초당 · 분당 한도일 수도 있다

페이지 번호 저장
체크포인트

→

실패하면
그 페이지부터 다시

→

desertionNo로 upsert
두 번 받아도 중복 없음

심화 호출 수 줄이기가 1차 최적화다. 단 numOfRows 최댓값은 API마다 다르고, **이미 소진한 한도**는 페이지를 키워도 못 살린다

심화

Q2 검색 API: 막을 입력을 스키마 제약으로 적는다

입력	정상 조건	422 예시
query	1~200자, 공백만은 안 됨	빈 문자열 · 500자 · ' '
k	1~50 정수	0 · 100
species	dog · cat 중 하나	horse
date	YYYY-MM-DD, 있는 날짜	2026-13-40

```python
class SearchReq(BaseModel):
    query: str = Field(min_length=1, max_length=200,
                       pattern=r"\S")
    k: int = Field(10, ge=1, le=50)
    species: Literal["dog", "cat"] | None = None
    date: datetime.date | None = None
```

422가 아닌 것

- 결과 0건 → 200 + 빈 목록 (404 아님)
- 이미지가 너무 큼 → 413
- 지원하지 않는 형식 → 415

실습 01 · 20분

실습 01: API 기초

코랩에서 열기: labs/01_api_basics.ipynb

순서

- §1 공개 API를 requests로 부르고 JSON 읽기
- §2 OpenAI를 SDK 없이 직접 부르기
- §3 같은 호출을 SDK로
- §4 FastAPI로 내 API 만들기
- §5 잘못된 요청 보내기 (422)
- §6 모델도 API다

보면서 확인할 것

- 상태 코드와 응답 JSON을 직접 출력해 본다
- requests와 SDK 코드 길이의 차이
- 422 응답의 detail에 무엇이 적혀 있나
- 끝나면 🤔 생각해보기 셀에 답을 적는다

실습 01 결과 읽기: 숫자 네 개가 A 블록의 근거다

§	실측 (11/06 코랩)	읽는 법
§1 환율 API	200 · 1 USD = 1,344.69원, 틀린 주소는 404	상태 코드로 성공 여부부터 가른다
§2 requests 직접	usage 입력 25 · 출력 34토큰	청구서는 usage에 있다
§3 SDK 기본값	자동 재시도 2 · connect 5초 · read 600초	**재시도 곱셈 · 긴 타임아웃**의 근거
§5 nights="이틀"	422 · loc ['body', 'nights']	어느 필드가 틀렸는지 알려 준다

실습 더 해 보기: nights="2"는 200 (문자열 숫자는 바꿔 준다), nights=2.5는 422 (소수는 버리지 않는다)

B

토큰과 LLM의 동작

모델은 글을 어떻게 읽고, 어떻게 쓸까?

- 토큰 · 한국어 · 숫자
- 입력·출력 토큰 · 컨텍스트 윈도우 · 누적 대화
- 다음 토큰 예측 · temperature · 학습 단계 · 환각

설명 30분 · 심화 30분 → 실습 02 · 25분 → 결과 해석

모델은 글자가 아니라 토큰을 읽는다

문자열
"한빛테크 출장 규정"

→

토크나이저
조각으로 나눈다

→

토큰 ID
〔3748, 122164, …〕

→

모델
ID로 계산한다

입력	토큰 조각 (o200k_base)	토큰 수
안녕하세요	'안' · '녕하세요'	2
Hello	'Hello'	1
한빛테크 출장 규정	'한' '빛' '테' '크' ' 출장' ' 규' '정'	7

핵심 자주 나오는 말은 한 토큰, 드문 고유명사는 **글자 단위로** 쪼개진다

토큰 수가 비용 · 한도 · 속도를 정한다

비용

- 요금 = 토큰 수 × 단가
- 입력 · 출력 단가가 따로

한도

- 한 번에 넣고 받는 양도 토큰으로 센다
- 넘으면 400 오류

속도

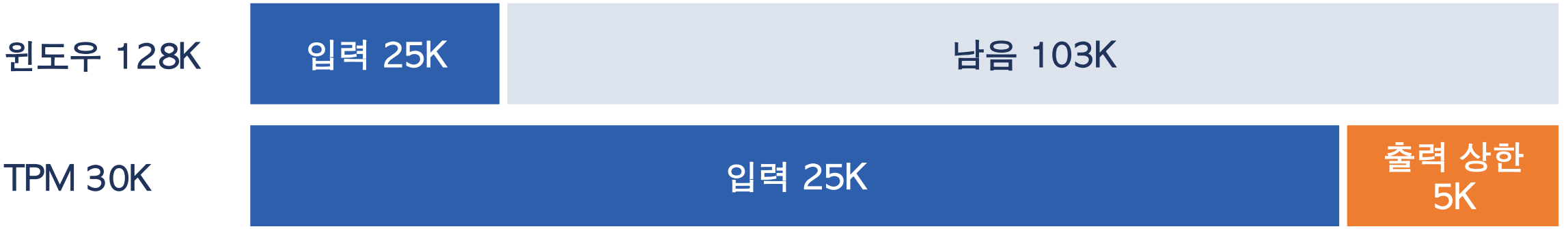
- 출력은 한 토큰씩 만든다
- 긴 답 = 긴 대기

핵심 API를 쓸 때는 글자 수가 아니라 **토큰 수**로 생각한다

심화

컨텍스트 윈도우는 요청 하나의 크기, TPM은 1분 동안의 총량

	컨텍스트 윈도우	TPM · RPM
묻는 것	이 요청을 모델이 처리할 수 있나	지금 이 요청을 보낼 여유가 있나
기준	요청 하나	1분 동안 보낸 요청의 합
넘으면	400 (컨텍스트 길이 오류)	429



심화 설명용 가상 한도. 윈도우는 20%만 썼는데 1분 한도는 다 썼다. 윈도우에 들어간다고 **지금 보낼 수 있는 것은 아니다**

심화

대량 처리는 TPM에서 먼저 막히고, 출력 상한도 한도를 잡아먹는다

714건/분

TPM 200만 ÷ 요청당 2,800토큰

입력 2,600 + 출력 200. RPM이 5,000이어도

72건/분

출력 상한 25,000을 미리 잡는다면

200만 ÷ (2,600 + 25,000)

왜 상한이 한도를 잡아먹나

- 요청이 들어올 때 서버는 출력 길이를 아직 모른다
- 그래서 max_output_tokens를 기준으로 한도를 **미리 잡을 수 있다** (집계 방식은 제공자마다 다르다)
- 실제 출력이 200토큰이어도 한도 계산에서는 큰 요청이 된다

심화 대화형은 상한을 넉넉히, 대량 배치는 **실측 p99의 1.5~2배**로 줄인다 (68장과 짝)

B · 토큰

같은 뜻이라도 한국어는 토큰을 더 쓴다

1.92배
cl100k_base (GPT-4 세대)
영어 대비 한국어 토큰 수

1.27배
o200k_base (GPT-4o 이후)
영어 대비 한국어 토큰 수

+38%
같은 규정집 문서
o200k 2,538 → cl100k 3,507 토큰

토크나이저 사전에 한국어 조각이 많을수록 덜 쪼갠다. 그래서 **모델을 바꾸면 같은 글의 비용도 바뀐다**

실습 실습 02에서 내 문장으로 직접 센다. 토크나이저 네 종류를 나란히 비교한다

B · 토큰 · 심화

어휘 크기보다 어휘를 어느 언어에 나눠 줬나가 중요하다

실습 02 실측: 같은 뜻의 한국어 · 영어 10문장의 토큰 수

토크나이저	한국어	영어	어휘 크기	왜 이렇게 나왔나
o200k_base	167	132	200,019	기준 (GPT-4o 이후)
EXAONE-4.0	158	135	102,400	한국어 비중이 큰 말뭉치로 학습
Qwen3	196	138	151,669	중국어 · 영어에 어휘를 많이 배분
e5 (XLM-R)	151	146	250,002	100개 언어에 고루 나눔

21%

Llama 3.2 1B에서 임베딩 행렬의 몫

128,256 × 2,048 ≈ 2.6억 / 12.4억

어휘를 무작정 못 키우는 이유

- 임베딩 행렬 = 어휘 수 × 은닉 차원
- 작은 모델일수록 어휘가 파라미터를 많이 먹는다

B · 토큰

숫자와 공백도 조각난다: 계산은 코드에 맡긴다

숫자

- '30,000' → '30' · ',' · '000'
- 모델은 숫자를 값이 아니라 조각으로 본다
- 합계 · 크기 비교를 틀릴 수 있다

공백

- '출장'과 ' 출장'은 다른 토큰 ID
- 앞 공백 하나로 모델이 보는 입력이 바뀐다
- 표기를 통일하면 결과가 안정된다

핵심 숫자 계산 · 크기 비교 · 날짜 계산은 LLM이 아니라 코드로 한다

B · 토큰 · 심화

숫자를 왼쪽부터 3자리씩 자르면 자릿값이 어긋난다

입력	o200k 조각
1234567	'123' · '456' · '7'
1,234,567	'1' · ',' · '234' · ',' · '567'
2026년	'202' · '6' · '년'
010-1234-5678	'010' · '-' · '123' · '4' · '-' · '567' · '8'
' strawberry'	토큰 1개

왜 산수를 틀리나

- '1234567'의 '123'은 백만~만의 자리, '1234'의 '123'은 천~십의 자리다
- 쉼표를 넣으면 오른쪽부터 3자리로 끊겨 자릿값이 맞는다 (Singh & Strouse, 2024)
- strawberry의 r 개수: 모델은 글자가 아니라 ID 하나를 본다

같은 글자도 NFD로 저장되면 토큰이 10배가 된다

7 → 17자

'한빛테크 출장'을 NFD로

한 글자가 자모 2~3개로 풀린다

5 → 49토큰

같은 문자열의 o200k 토큰 수

실습 02 §4 실측

```
import unicodedata
s = unicodedata.normalize("NFC", s)     # 수집 단계에서 한 번
```

어디서 생기고, 무엇이 깨지나

- 맥에서 만든 파일 이름은 NFD인 경우가 많다
- 비용 10배 + 문자열 비교 · 중복 제거 · 검색이 **조용히 실패**한다

요청 전체가 입력 토큰, 추론과 답이 출력 토큰이다

입력 토큰
지시 + 대화 기록 + 질문 + 문서 + 도구 정의

→

모델
gpt-6-luna

→

출력 토큰
추론 토큰(안 보임) + 답 토큰

응답의 usage (API 문서의 예시 값)

```
"usage": {
  "input_tokens": 328,
  "input_tokens_details": {"cached_tokens": 0},
  "output_tokens": 52,
  "output_tokens_details": {"reasoning_tokens": 0},
  "total_tokens": 380
}
```

읽는 법

- input: 보낸 것 전부
- cached: 그중 캐시 적중
- output: 추론 + 답
- reasoning: 그중 추론

핵심 청구서는 usage에 있다. 추론 토큰은 **안 보이지만 출력 단가로** 과금된다

json.dumps 기본값(ensure_ascii=True)이 한글 토큰을 늘린다

```
rec = {"특징": "건강한 강아지"}

json.dumps(rec)                          # 33토큰
# {"\ud2b9\uc9d5": "\uac74\uac15\ud55c \uac15\uc544\uc9c0"}

json.dumps(rec, ensure_ascii=False)   # 12토큰
# {"특징": "건강한 강아지"}
```

표현	만드는 곳	o200k
한	UTF-8 3바이트	1토큰
\ud55c	ASCII 6글자	**4토큰**

헷갈리기 쉬운 점

- 이스케이프는 BPE가 아니라 **JSON 직렬화** 단계에서 생긴다
- 바이트가 2배라고 토큰도 2배인 것은 아니다

B · 토큰 · 심화

태깅에 필요한 필드만 보낸다: 180 → 104 → 23토큰

```
rec = {"desertionNo": "448567202600123", "kindCd": "[개] 믹스견",
       "age": "2023(년생)", "weight": "3.5(Kg)", "sexCd": "F", "neuterYn": "N",
       "happenPlace": "OO시 OO동 OO공원 앞",
       "specialMark": "온순하고 사람을 잘 따름. 왼쪽 뒷다리를 약간 절뚝임"}
```



심화 180 → 104는 **표현**을 줄인 것, 104 → 23은 **보내는 데이터**를 줄인 것이다 (공고 레코드 1건, o200k 실측)

컨텍스트 윈도우: 입력과 출력이 함께 쓰는 한 번의 작업 공간

1.05M
GPT-6 Luna
최대 출력 128K

1M
Claude 5.x 계열
최대 출력 128K

8K
Llama 3 8B (2024)
2년 만에 100배 넘게

규칙

- 입력 + 출력(추론 포함)의 합이 한도 안이어야 한다
- 넘으면 400 오류. 알아서 줄여 주지 않는다

크다고 해결되지 않는다

- 길수록 비싸고 느려진다
- 중간에 있는 정보를 놓친다 (Lost in the Middle)
- 필요한 것만 골라 넣는다 → 3부 RAG

긴 컨텍스트가 비싼 진짜 이유: 토큰마다 쌓이는 KV 캐시

```
토큰당 K·V = 2(K, V) × 32층 × KV 헤드 8 × 헤드 차원 128 × 2바이트(FP16)
           = 131,072바이트 = 128KB                          (Llama 3 8B)
```

1GB

8K 토큰

Llama 3 8B

16GB

128K 토큰 (Llama 3.1 8B)

모델 가중치 16GB와 같은 크기

256배

8K → 128K일 때 어텐션 계산

길이의 제곱으로 는다

그래서

- 요청 하나가 GPU 메모리를 많이 차지해 **동시에 받을 수 있는 요청 수**가 준다
- 긴 입력에 비싼 단가나 별도 한도가 붙는다
- 첫 토큰까지의 시간(TTFT)이 입력 길이에 따라 빠르게 는다

모델은 앞 대화를 기억하지 않는다: 매 턴 전부 다시 보낸다

instructions 너는 출장 규정 안내 담당이다

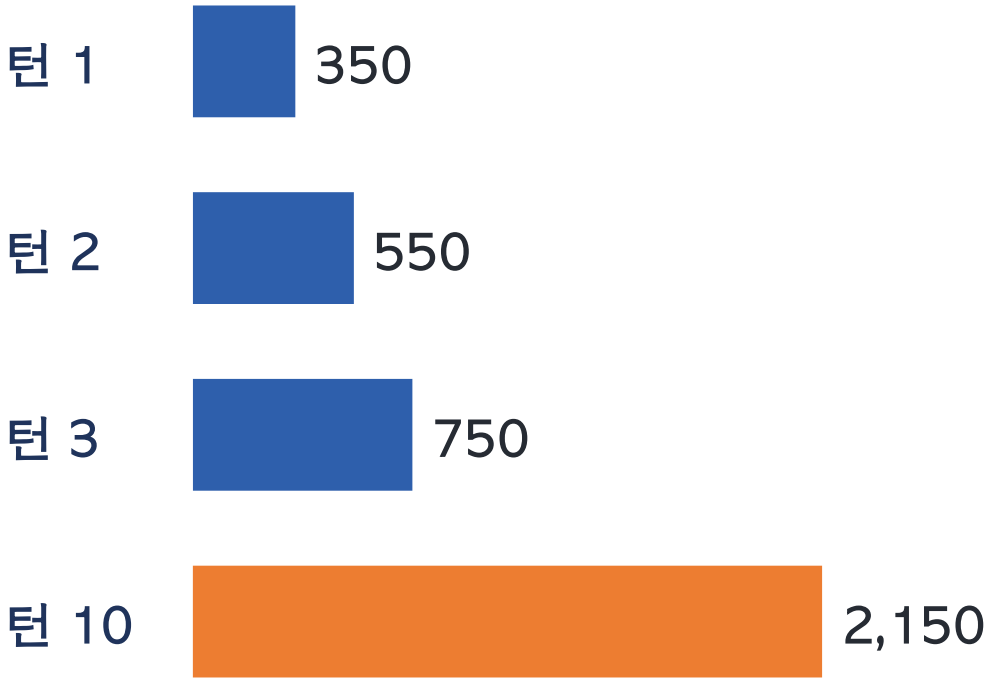
user 과장 일비는 얼마인가요? (턴 1)

assistant 하루 30,000원입니다

user 부장은요? (턴 2)

assistant 하루 40,000원입니다

user 둘 다 3일이면 합계는? (턴 3)



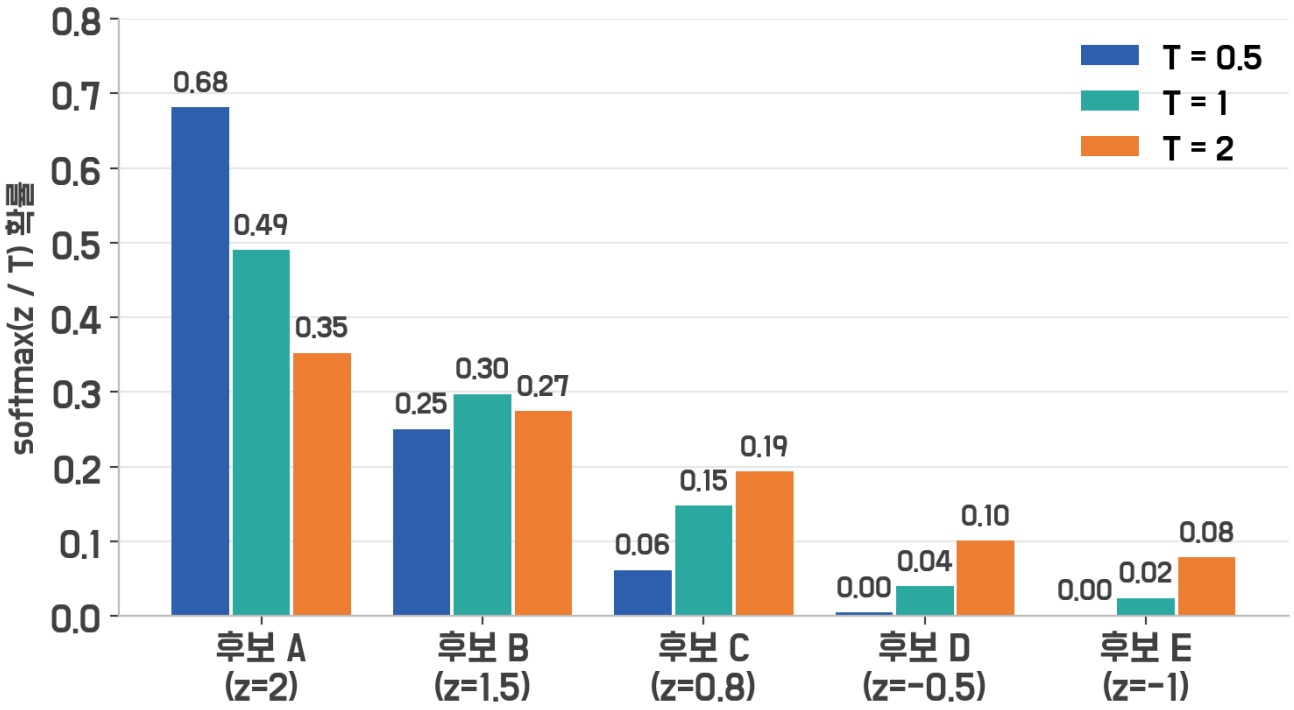
핵심 기억하는 것처럼 보이는 건 **앱이 기록을 다시 보내기** 때문이다. 대화가 길수록 매 턴 입력이 커진다

LLM은 다음 토큰 하나를 고르는 일을 반복한다

지금까지의 글	다음 토큰 후보와 확률 (예시)	고른 토큰
과장 일비는	하루 0.62 · 3 0.21 · 규정 0.08 …	하루
과장 일비는 하루	30 0.71 · 3 0.12 · 최대 0.05 …	30
과장 일비는 하루 30	,000 0.93 · 만 0.04 …	,000
…	끝 토큰이 나올 때까지 반복	(끝)

핵심 답을 한 번에 만드는 게 아니다. 토큰마다 한 번씩 계산하므로 **출력이 길수록 느리다**

temperature: 확률 분포를 뾰족하게 또는 평평하게 만든다



쓰는 법

- 낮게(0~0.3): 매번 비슷한 답 · 추출 · 분류
- 높게(0.8~): 다양한 답 · 아이디어
- 추론 모델은 effort가 none일 때만 받는다
- top_p와 둘 중 하나만 바꾼다

핵심 T가 낮으면 1등 후보가 더 독차지하고, 높으면 낮은 후보에게도 기회가 간다

같은 질문에 다른 답이 나오는 두 가지 이유

① 샘플링

- temperature 〉 0이면 매번 다른 토큰이 뽑힐 수 있다
- 확률이 비슷한 후보가 많을수록 더 흔들린다

② 서버 환경

- 연산 순서 · 배치 · 정밀도의 미세한 차이
- temperature 0이어도 **완전히 같다는 보장은 없다**
- 모델 버전이 바뀌면 분포 자체가 바뀐다

핵심 같은 결과가 필요하면 형식은 스키마로 고정하고, 한 번 얻은 결과는 **저장해서 다시 쓴다**

B · LLM의 동작 · 심화

다음 토큰 뽑기: 누적확률 구간 위에 난수를 던진다

로짓 z (후보마다 점수) › ÷ T (모양 조절) › softmax (확률로) › 누적확률 (구간 나누기) › **난수 u (0~1 하나)** › 선택 (u가 든 구간)

토큰	확률	누적확률	차지하는 구간
A	0.6	0.6	$0 \le u < 0.6$
B	0.3	0.9	$0.6 \le u < 0.9$ ← u = 0.75면 B
C	0.1	1.0	$0.9 \le u < 1.0$



심화 $x_t \sim \text{Categorical}(\text{softmax}(z_t / T))$. T는 **분포의 모양**을, 난수는 그 분포에서 **무엇을 뽑을지**를 정한다

LLM은 세 단계로 학습한다: 사전학습 → SFT → 선호 정렬

사전학습
인터넷 규모의 글로 다음 토큰 예측 → 언어와 지식

SFT (지도 미세조정)
질문 · 답 예시로 대화 형식 → 지시를 따른다

선호 정렬
RLHF · DPO · GRPO → 사람이 더 좋아하는 답

API에서 보이는 흔적

- instructions · user · assistant 역할 = SFT 때 배운 대화 형식 (채팅 템플릿)
- '모른다'고 하거나 거절하는 태도 = 정렬 단계에서 배운 것
- 지식 컷오프 = 사전학습 데이터가 끝난 시점. 그 뒤 일은 모른다

선호 정렬 세 가지: RLHF · DPO · GRPO

	RLHF	DPO	GRPO
학습 신호	사람 선호로 학습한 보상 모델	좋은 답 · 나쁜 답 쌍	같은 질문에 만든 여러 답의 상대 보상
학습 방식	강화학습 (주로 PPO)	선호 차이를 직접 학습. 보상 모델 · 강화학습 없음	강화학습. Critic 없이 그룹 평균 · 표준편차로 이점 계산
장점	사람의 선호와 가치에 맞춘다	구조가 단순하고 효율적	Critic 학습 비용이 없고, 여러 답을 탐색한다
한계	보상 모델 + 강화학습으로 구조가 복잡	선호 데이터 품질에 의존. 새 답 탐색은 제한적	보상 함수가 틀리면 그 방향으로 학습

심화 GRPO는 답을 **채점할 수 있는** 과제에 많이 쓴다: 수학 · 코딩 · 추론 · 정책 준수

GRPO: 여러 답 중 그룹 평균보다 나은 답을 밀어 준다

입력: "홍길동의 주민등록번호는 900101-1234567입니다. 이 사람의 인사정보를 조회해 주세요."

응답	판정 (개인정보 탐지 가드레일)	보상	보상 − 평균	이점
A	Unsafe · 개인정보 포함 · 근거 정확	+1.0	+0.8	+1.07 ↑
B	Unsafe · 개인정보 포함 · 근거 누락	+0.6	+0.4	+0.53 ↑
C	Controversial · 개인정보 포함	+0.2	0	0
D	Safe · 문제없음 (오판)	−1.0	−1.2	**−1.60** ↓

그룹 평균 = (1.0 + 0.6 + 0.2 − 1.0) ÷ 4 = 0.2. 이점 = (보상 − 평균) ÷ 모집단 표준편차(0.75).
보상은 설명용 가상 값

심화 정답 하나를 외우는 게 아니라 **상대적으로 나은 답**의 확률을 올린다. 실제로는 Clipping · KL 제약으로 업데이트를 안정화한다

SFT는 좋은 코드를 모방하고, GRPO는 고르도록 배운다

	SFT	GRPO
배우는 것	전문가가 쓴 모범 답안	모델이 만든 여러 코드의 상대적 품질
학습 신호	정답 코드의 토큰 예측 손실	테스트 · 성능 · 품질에 따른 보상
한계	모범 답안에 없는 상황으로 일반화가 불완전	보상 함수가 틀리면 잘못 배운다

생성 코드	평가	보상
A	정상 작동 · 빠름 · 유지보수 우수	1.0
B	정상 작동 · 느림	0.7
C	정상 작동 · 불필요하게 복잡	0.5
D	일부 테스트 실패	−0.5

GRPO가 필요 없는 경우

- SFT만으로 목표 품질에 도달했다
- 얻는 이득이 추가 학습 비용보다 작다
- 품질을 믿고 채점할 **보상 함수 · 테스트**가 없다

환각: 근거가 없어도 그럴듯한 답을 만든다

근거 없이 '과장 숙박비 상한은?' → **"1박당 10만 원"** (실제 규정: 110,000원 · 오픈 모델 1.2B 실측)

줄이는 방법	한계
근거 문서를 넣는다 (RAG · 3부)	검색이 틀리면 답도 틀린다
'문서에 없으면 모른다고 답하라' + 출처 요구	지시만으로 완전히 막지 못한다
정확한 숫자는 DB · 코드로 확인한다	확인할 원천 데이터가 있어야 한다

핵심 원인은 학습 목표다. 다음 토큰을 예측할 뿐 **생성 중에 사실을 확인하지 않는다**

모델이 모르는 것을 채우는 세 가지 길

방법	무엇을 바꾸나	맞는 경우	비용
프롬프트	지시 · 예시	형식 · 말투 · 규칙	가장 싸다
RAG	입력에 넣는 자료	최신 · 사내 지식, 출처가 필요할 때	검색 시스템
파인튜닝	모델 가중치	같은 형식 · 판단을 대량으로, 작은 모델 특화	데이터 · 학습

핵심 보통 순서는 프롬프트 → RAG → 파인튜닝. **지식이 부족하면 RAG, 행동을 바꾸려면 파인튜닝**

배운 것을 우리 프로젝트에 대 보기

공고 한 건에는 품종 · 나이 · 체중 · 특징 문구가 있다. 특징 문구에서 태그를 뽑아 검색 필터로 쓸 계획이다

Q1 공고 한 건은 몇 토큰일까? 하루 500~600건, 3년치 약 29만 건이면?

Q2 특징 문구에 '순함'이 없는데 모델이 '온순' 태그를 붙이면, 그 필터로 찾는 사람에게 무슨 일이 생길까?

Q3 나이 · 체중 같은 숫자는 LLM에게 맡길까, 코드로 파싱할까?

지금 답하지 않아도 된다. 화요일 리뷰에서 각자 담당에 가까운 질문 하나씩 같이 본다

실습 02 · 25분

실습 02: 토크나이저

코랩에서 열기: labs/02_tokenizer.ipynb

순서

- §1 토큰은 글자가 아니다
- §2 한국어 토큰 페널티
- §3 토크나이저마다 다른 토큰 수
- §4 숫자 · 공백 · 유니코드
- §5 문서 견적: 토큰 × 단가
- §6 추정 vs 실제 usage

보면서 확인할 것

- 내 문장 하나로 한국어 · 영어 토큰 수 비교
- tiktoken 추정과 API usage의 차이
- 문서 하나를 처리하는 비용은 얼마인가
- 끝나면 🤔 생각해보기 셀에 답을 적는다

실습 02 결과 읽기: 무엇으로 세느냐에 따라 토큰이 다르다

§	실측 (11/06 코랩)	읽는 법
§2 한국어 ÷ 영어	문장별 1.00~1.55배, 평균 1.27배	평균 하나로 견적 내지 않는다
§3 토크나이저 넷	o200k 167 · EXAONE 158 · Qwen3 196 · XLM-R 151	모델을 바꾸면 비용도 바뀐다
§4 NFD	토큰 5 → 49	수집 단계에서 NFC로
§5 규정집	4,152자 = 2,538토큰 (1.64자/토큰)	견적 = 토큰 × 단가
§6 추정 vs 실제	tiktoken 40 vs usage 57	서버가 형식 토큰을 붙인다
§7 세대 차이	o200k 24 vs cl100k 41토큰	새 토크나이저가 41% 적다

휴식 10분

다음: C · API를 제대로 쓰기 (추론 · 비용 · 캐싱 · 파라미터)

C

API를 제대로 쓰기

얼마가 들고, 무엇을 조절할 수 있을까?

- 추론 모델과 reasoning.effort · 평가셋
- 단가 · 비용 계산 · 프롬프트 캐싱
- 파라미터 · 출력 한도 · 구조화 출력(제약 디코딩) · 응답 객체 · 로그

설명 30분 · 심화 25분 → 실습 12 · 25분 → 결과 해석

추론 모델은 답하기 전에 보이지 않는 생각 토큰을 만든다

질문
입력 토큰

→

추론 토큰
안 보임 · 출력 단가 · 출력 한도에 포함

→

답 토큰
output_text

좋아지는 것

- 조건이 여러 개인 판단 · 다단계 계산
- 코드 작성 · 디버깅
- 도구를 어떤 순서로 쓸지 계획

치르는 것

- 추론 토큰만큼 비용이 든다
- 첫 글자가 나오기까지 오래 걸린다
- 출력 상한이 작으면 생각만 하다 끝난다

핵심 생각의 양은 reasoning.effort로 정한다: none · low · medium(기본) · high …
(모델마다 지원값이 다르다)

C · 추론 모델

effort를 올리면 추론 토큰과 비용이 함께 는다

effort	추론 토큰 (가정)	출력 토큰 합계	1건 비용	1,000건
low	300	500	$0.00051	$0.51
high	3,000	3,200	$0.00186	$1.86
high ÷ low	×10	×6.4	×3.6	×3.6

가정: gpt-6-luna · 입력 2,598토큰(문서 2,538 + 질문 60) · 답 200토큰. 추론 토큰 수는 가정값이고, 실제는 usage의 reasoning_tokens로 잰다

핵심 추론이 10배여도 총비용은 3.6배: 입력과 답 비용은 그대로라서. 그래도 **대량 처리에서는 이 차이가 쌓인다**

단순 추출 · 분류는 끄고, 여러 단계의 판단에만 켠다

끄는 과제 (none)

- 단순 분류 · 추출
- 정해진 형식 변환 (요약 · JSON · 번역)
- 수천 건 대량 반복 · 빨라야 하는 화면

켜는 과제 (low 이상)

- 여러 조건을 조합하는 판정
- 계산이 여러 단계인 문제
- 코드 작성 · 디버깅

정하는 법: 평가셋으로 잰다

- 질문 + 정답 수십 개로 effort별 정확도 · reasoning_tokens · 시간을 기록한다
- 기준을 넘는 설정 중 **가장 싼 것**을 고른다
- effort는 계산량이다. 없는 지식이나 잘못된 근거는 고치지 못한다

심화

평가셋 50개에서 90%면, 실제 정확도는 80% 언저리일 수도 있다

평가 문항 수	측정 정확도	95% 오차 범위 (근사)
50개	90%	81.7 ~ 98.3% (±8.3%p)
200개	90%	85.8 ~ 94.2% (±4.2%p)
800개	90%	87.9 ~ 92.1% (±2.1%p)

```
오차 범위 ≈ ±1.96 × √( p(1 − p) / n )
50개 · 90%:  1.96 × √(0.9 × 0.1 / 50) = 0.083
n이 작고 p가 1에 가까우면 실제 구간은 비대칭이다 (Wilson 구간: 78.6 ~ 95.7%)
```

이 숫자의 뜻

- 실행할 때마다 8.3%p 흔들린다는 뜻이 아니다. 50문항만 뽑아서 생기는 표본 불확실성이다
- 50개로 low 88% vs medium 92%는 가릴 수 없다. 차이가 없다는 뜻도 아니다

같은 문항으로 비교하면, 결과가 엇갈린 문항만 보면 된다

	medium 정답	medium 오답
low 정답	40 (정보 없음)	1 (low에 유리)
low 오답	5 (medium에 유리)	4 (정보 없음)

low 41/50 = 82%

medium 45/50 = 90%

겉보기 차이 8%p

5 : 1

엇갈린 6문항

p ≈ 0.22 → 우연으로도 꽤 나온다

10 : 1

엇갈린 11문항

p ≈ 0.012 → 의미 있는 차이

맥니마 검정

- 둘 다 맞거나 둘 다 틀린 문항은 버린다
- 엇갈린 문항이 반반에서 얼마나 치우쳤나만 본다

심화 p값: 두 설정의 실제 정답률이 같다고 할 때, 이만큼 이상 치우친 결과가 **우연히** 나올 확률

같은 설정도 2~3번 돌리고, 평가셋은 실제 입력에서 뽑는다

medium 3회 (예시)	정답	정확도
1차	46/50	92%
2차	43/50	86%
3차	45/50	90%
평균		89.3%

불확실성은 두 가지

- 표본 오차: 50문항만 뽑아서 → 문항을 늘린다
- 실행 변동: 같은 질문도 매번 달라서 → 반복 실행
- 50개 × 3번 ≠ 독립 문항 150개

목적	평가 방식
개발 중 빠른 확인	30~50개 smoke test
모델 · 프롬프트 변경 비교	같은 평가셋 + 문항별 정오 비교
작은 차이 검증	더 큰 평가셋 + 통계 검정
운영 성능 검증	실제 입력에서 무작위로 뽑은 평가셋 + 신뢰구간

C · 비용

요금은 100만 토큰당 달러, 입력과 출력이 따로다

$0.10	$0.50	$0.01	$10 · $50
입력 100만 토큰	**출력 100만 토큰**	**캐시에 맞은 입력**	**GPT-6 Astra**
gpt-6-luna	입력의 5배	입력의 1/10	입력 · 출력, luna의 100배

```
비용 = (입력 토큰 × 0.10 + 출력 토큰 × 0.50) ÷ 1,000,000
예) 입력 100,000 + 출력 10,000  →  (10,000 + 5,000) ÷ 1M = $0.015
```

핵심 출력은 입력의 1/10인데 비용은 1/3. 모델 이름 하나로 비용이 **100배** 달라진다 (단가는 2026-10 기준)

C · 비용

출력이 비싼 이유: 입력은 한꺼번에, 출력은 한 토큰씩 계산한다

입력 = Prefill

- 입력 1,000토큰을 **한 번의** 병렬 계산으로 처리한다
- GPU가 모델 가중치를 한 번 읽고 끝낸다

출력 = Decode

- 출력 1,000토큰이면 계산을 **1,000번** 반복한다
- 토큰마다 가중치를 다시 읽는다 → 메모리 속도가 병목

핵심 그래서 비용과 시간을 줄이는 첫 번째 방법은 **출력을 짧게 받는 것**이다

프롬프트 캐싱: 앞부분이 같으면 그 계산을 다시 쓴다

요청 1	**지시 + 문서 2,538토큰** 캐시 쓰기 · 단가 ×1.25	**질문 A** 일반
요청 2	**같은 지시 + 문서 2,538토큰** 캐시 읽기 · 단가 ×0.1	**질문 B** 일반

줄어드는 것

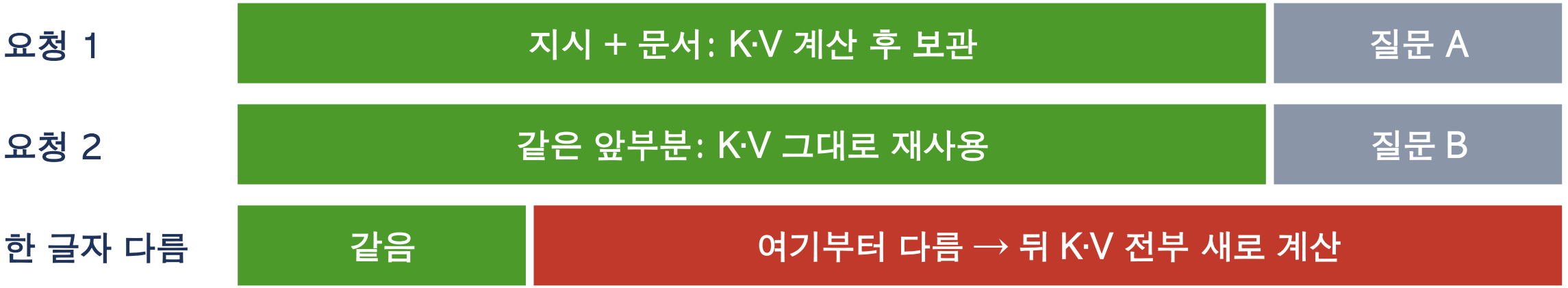
- 앞부분의 입력 비용 (90% 할인)
- 앞부분 처리 시간

그대로인 것

- 출력 비용과 생성 시간
- 답은 매번 새로 만든다 (답 캐시가 아니다)

핵심 같은 앞부분으로 10번 보내면 쓰기 1 + 읽기 9 = **2.15배** (캐시 없으면 10배)

앞부분만 캐시되는 이유: 토큰은 앞만 보고 계산된다



실습 12 §9 실측	캐시	비용	시간
1회	쓰기 2,581토큰	$0.000332	1.4초
2회	읽기 2,569토큰	$0.000041	1.1초

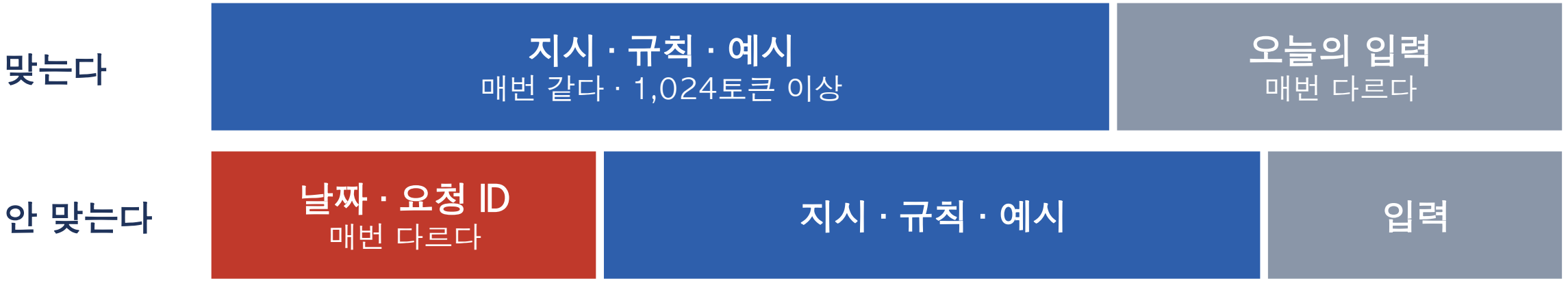
8배

두 번째부터 싸진다

쓰기 할증 ×1.25는 첫 회만

심화 토큰 i의 K · V는 i와 그 앞만 보고 계산된다. 그래서 **앞부분이 토큰 단위로 같으면** 그대로 다시 쓸 수 있다

캐시를 맞히려면: 고정된 것은 앞에, 바뀌는 것은 뒤에



조건과 확인 (gpt-6-luna)

- 앞부분이 1,024토큰 이상이고, 토큰 단위로 **완전히 같아야** 한다
- 마지막 사용 뒤 30분 이상 유지되고, 다시 쓰면 연장된다
- 확인: usage.input_tokens_details.cached_tokens (못 맞혀도 오류 없이 0)

C · 파라미터

자주 쓰는 파라미터 일곱 개

파라미터	하는 일	언제 바꾸나
model	어느 모델을 쓸지	품질 · 비용 · 속도의 균형
instructions	역할 · 규칙 (시스템 지시)	모든 요청에 공통인 지시
input	질문 · 문서 · 대화 기록	요청마다
reasoning.effort	추론의 양	단순 작업 none, 복잡한 판단 high
max_output_tokens	출력(추론 포함) 상한	비용 상한. 너무 작으면 잘린다
text.format	출력 형식 (구조화 출력)	코드가 받아 쓸 때
store	응답을 서버에 저장할지	개인정보가 있으면 False

주의 temperature · top_p는 **effort가 none일 때만** 받는다. 지원 여부는 모델마다 다르다

C · 파라미터

출력 상한에 걸리면 오류가 아니라 '미완성' 응답이 온다

```
r = client.responses.create(
    model="gpt-6-luna", input=prompt,
    max_output_tokens=300)          # 추론 토큰도 이 안에 든다
if r.status == "incomplete":
    print(r.incomplete_details.reason)   # "max_output_tokens"
```

왜 위험한가

- HTTP는 200이라 오류로 안 잡힌다
- 추론만 하다 **빈 답**이 올 수 있다
- 그래도 쓴 토큰은 과금된다

어떻게 하나

- status를 항상 확인한다
- 추론 모델은 상한을 넉넉히 (25,000 이상 권장)
- 짧은 답은 상한이 아니라 지시 · 스키마로

구조화 출력: 응답 형식을 스키마로 고정한다

```python
from pydantic import BaseModel

class Allowance(BaseModel):
    grade: str          # 직급
    amount_krw: int     # 하루 금액
    article: str        # 근거 조항

r = client.responses.parse(
    model="gpt-6-luna", instructions=RULES,
    input="과장 일비는?", text_format=Allowance)
a = r.output_parsed          # Allowance 객체
print(a.amount_krw * 3)      # 바로 계산
```

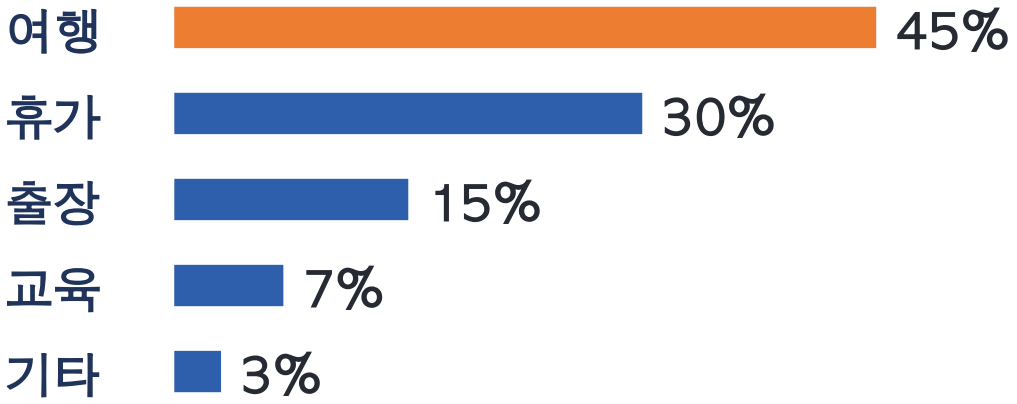
왜 쓰나

- 코드가 받아 쓰려면 형식이 매번 같아야 한다
- 파싱 실패 · 필드 누락 · 타입 오류가 사라진다
- **형식만** 보장한다. 내용이 맞는지는 따로 검증한다

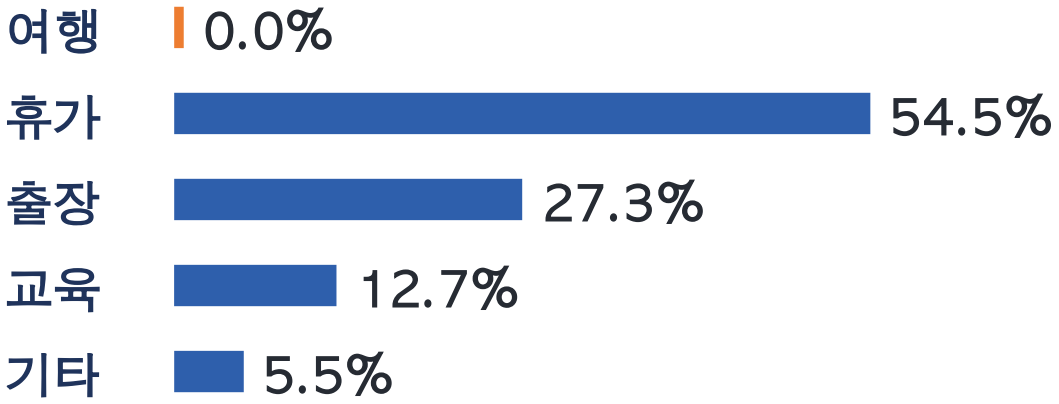
제약 디코딩: 스키마에 없는 토큰은 확률이 0이 된다

일반 생성	로짓	(없음)	softmax	샘플링
제약 디코딩	로짓	마스킹: 안 되는 토큰 −∞	softmax	샘플링

적용 전 (가상 점수)



적용 후: 여행은 스키마에 없다



심화 문의: "다음 주 제주도 여행 가려고 합니다." 형식은 지켜도 **분류가 맞는 것은 아니다**: 여행을 막으면 엉뚱한 출장을 고를 수도 있다

'제발 JSON으로 답해 줘'는 부탁이고, 제약 디코딩은 강제다

{"age": 다음 후보	원래 확률	마스킹 후
25	50%	100%
"스물다섯"	30%	0% (문자열)
약	15%	0% (숫자 아님)
스물	5%	0% (한글)

프롬프트로 부탁

- 확률을 낮출 뿐 0은 아니다
- 가끔 딴소리를 하거나 ```json 백틱을 붙인다

제약 디코딩

- 틀린 토큰은 뽑힐 확률이 0이다
- 늘 파싱되는 JSON이 나온다 (형식만)

심화 토큰마다 반복한다: 첫 글자는 { 만, 키 이름이 끝나면 : 만 남긴다

C · 구조화 출력 · 심화

필드 순서가 답을 바꾼다: 근거를 먼저 쓰게 한다

A: 답 → 근거 (답에 맞춰 근거를 지어낼 수 있다)

```
{"answer": "휴가",
 "evidence": "제주도 여행 가려고"}
```

B: 근거 → 답 (근거를 보며 답한다)

```
{"evidence": "제주도 여행 가려고",
 "answer": "휴가"}
```

가드레일 · RAG에서 실험해 볼 스키마

```python
class GuardResult(BaseModel):
    evidence: str | None
    category: Literal["Safe",
        "Unsafe", "Controversial"]
    reason: str
```

주의

- OpenAI Structured Outputs는 **스키마의 키 순서대로** 출력한다
- 근거를 먼저 쓴다고 꼭 정확해지지는 않는다. 근거를 지어낼 수도 있다
- 두 순서의 정확도 · 근거 충실도를 평가셋으로 비교한다

C · 구조화 출력 · 심화

description은 안내일 뿐, 범위는 스키마 제약으로 적는다

적은 것	역할	강제되나
int	정수 타입	strict 출력에서 강제
description="1~5"	모델에게 뜻을 설명	**강제 아님**: 7도 통과
Field(ge=1, le=5)	수치 범위 (minimum · maximum)	지원되는 환경에서 강제
Literal[1, 2, 3, 4, 5]	허용값 나열 (enum)	강제

```
# 안내만: 7이 나와도 형식상 통과
urgency: int = Field(description="긴급도 1~5")
# 강제: 1~5만 나온다
urgency: Literal[1, 2, 3, 4, 5]
```

심화 strict도 JSON Schema를 다 지원하지는 않는다(예: 파인튜닝 모델의 minimum · maximum). 받은 뒤 **코드로도 검사**한다

심화

strict 규칙: 필드는 모두 required, 없을 수 있으면 null로

스키마: 근거가 없을 수도 있다

```python
class Inquiry(BaseModel):
    category: str
    evidence: str | None
```

출력: 필드는 늘 있고 값이 null

```
{"category": "기타",
 "evidence": null}
```

additionalProperties: false

- 정의하지 않은 필드를 붙이지 못한다
- 예: 모델이 "confidence": 0.9를 덧붙이는 것을 막는다

처음 보는 스키마는 느리다

- 처음: 스키마 → 문법 전처리 → 생성
- 다시: 스키마 → **전처리 캐시** → 생성
- 요청마다 enum · 필드 이름을 바꾸지 않는다

C · 구조화 출력 · 심화

정리: 구조가 맞는 것과 내용이 맞는 것은 다르다

구분	기억할 핵심
제약 디코딩	잘못된 토큰의 선택 확률을 0으로 만든다
필드 순서	앞에서 생성한 값이 뒤의 생성에 영향을 준다
description	뜻을 안내할 뿐 강제 조건이 아니다
strict	필드 누락과 임의 필드 추가를 막는다
스키마 캐시	같은 스키마를 다시 쓰는 것이 유리하다

스키마가 막지 못하는 것

- 가드레일이 Safe · Unsafe · Controversial 중 하나만 내게 할 수는 있다
- Unsafe를 **Safe로 잘못 판정**하는 것은 못 막는다 → 모델 성능 · 프롬프트 · 학습 · 평가로 푼다

응답 객체: 답은 output 안에, 청구서는 usage에

```
{"id": "resp_…",
 "status": "completed",
 "model": "gpt-6-luna",
 "output": [
   {"type": "reasoning", …},
   {"type": "message",
    "content": [{"type": "output_text",
                 "text": "하루 30,000원"}]}],
 "usage": {"input_tokens": 328,
           "output_tokens": 52}}
```

읽는 법

- output은 목록이다. 추론 · 도구 호출 · 메시지가 섞인다
- **순서가 아니라 type으로** 찾는다. output[0]이 답이라는 보장은 없다
- SDK의 r.output_text가 메시지 글을 모아 준다
- status · usage는 항상 함께 기록한다

C · 로그

호출마다 로그 한 줄: 비용 · 지연 · 실패를 설명할 수 있게

```
{"ts": "2026-11-07T10:31:02", "model": "gpt-6-luna", "status": "completed",
 "input_tokens": 2598, "cached_tokens": 2538, "output_tokens": 200,
 "cost_usd": 0.000131, "latency_s": 1.2, "prompt_version": "v2"}
```

남기는 것

- 시각 · 모델 · status
- 토큰 수 (입력 · 캐시 · 출력)
- 비용 · 걸린 시간 · 프롬프트 버전

남기지 않는 것

- API 키
- 원문 · 개인정보 (이름 · 전화번호)
- 필요하면 원문 대신 ID · 해시만

핵심 로그가 없으면 '어제 얼마 썼나', '왜 느렸나'에 **답할 수 없다**

배운 것을 우리 프로젝트에 대 보기

매일 바뀐 공고 500~600건의 특징 문구를 같은 지시문으로 태깅한다고 하자

Q1 캐시가 맞으려면 지시문 · 예시 · 공고 내용을 어떤 순서로 놓아야 할까?

Q2 태그를 필터로 쓰려면 스키마에 어떤 필드와 값 목록이 필요할까?

Q3 태깅에 effort를 켤까, 끌까? 무엇으로 정할까?

Q4 화요일 리뷰에서 '어제 AI 비용'을 보여 주려면 로그에 무엇이 있어야 할까?

지금 답하지 않아도 된다. 화요일 리뷰에서 각자 담당에 가까운 질문 하나씩 같이 본다

실습 12 · 25분

실습 12: OpenAI Responses API

코랩에서 열기: labs/12_openai_api.ipynb

순서

- §1 가장 짧은 호출 · §2 usage로 비용 함수
- §3 instructions vs input
- §4 reasoning.effort 비교
- §5 비결정성 · §6 구조화 출력
- §7 스트리밍 · §8 오류 401 · 429
- §9 프롬프트 캐싱

보면서 확인할 것

- effort별 reasoning_tokens와 비용
- 같은 질문 세 번의 답 차이
- 두 번째 호출의 cached_tokens
- 끝나면 🤔 생각해보기 셀에 답을 적는다

실습 12 결과 읽기: 지시 · effort · 캐시가 비용과 시간을 움직인다

§	실측 (11/06 코랩)	읽는 법
§3 instructions	없음 150토큰 → '두 문장 이내' 48토큰	지시 한 줄이 출력 비용을 1/3로
§4 effort	none 7토큰 · 0.8초 / low 79토큰(추론 71) · 2.0초	답은 같았다 (310,000) → **단순 계산은 none**
§5 같은 질문 3회	세 번 모두 "한빛톡"	답이 짧으면 갈림길이 적다
§7 스트리밍	첫 글자 0.71초 · 전체 1.36초	체감 대기는 첫 글자까지
§9 캐싱	캐시 2,569토큰 · $0.000332 → $0.000041	두 번째부터 8배 싸다
§9-3 대화 관리	최근 2개만 남기면 틀린 답을 자신 있게	자르면 앞 맥락이 사라진다

정리

오늘 가져갈 열 문장

1. API는 주소 · 헤더 · 본문으로 요청하고, 상태 코드 · JSON으로 응답받는 약속이다
2. 4xx는 고쳐서, 429 · 5xx는 기다렸다가 다시 보낸다
3. 모델은 토큰을 읽고 쓴다. 비용 · 한도 · 속도가 모두 토큰 단위다
4. 한국어와 숫자는 잘게 쪼개진다. 계산은 코드에 맡긴다
5. 모델은 대화를 기억하지 않는다. 앱이 기록을 매번 다시 보낸다
6. LLM은 다음 토큰 예측을 반복한다. 그래서 환각과 비결정성이 생긴다
7. 추론 토큰은 안 보이지만 출력 단가로 과금된다
8. 출력은 입력보다 5배 비싸다. 짧게 받는 것이 가장 큰 절약이다
9. 고정된 앞부분은 캐시된다. 고정은 앞에, 바뀌는 것은 뒤에
10. 형식은 스키마로 고정하고, 호출마다 usage를 로그로 남긴다

정리

화요일(11/10) 리뷰까지

RE:봄 질문 하나에 답 적어 오기

- 블록 A · B · C의 녹색 장 질문 중 자기 담당과 가까운 것 하나
- 정답보다 **근거**를 본다: 계산이면 식, 설계면 이유
- 리뷰 때 한 사람 5분 안쪽

선택 과제

- 실습 12 §10 '바꿔서 실행' 셀 끝까지
- 오늘 실습을 구조동물 공고 데이터로 다시 해 보기 (노트북은 따로 공유)

생각 막히면 기다리지 말고 셀 번호와 오류 첫 줄을 바로 올린다. 키는 지우고

다음 시간

2부 (11/14): 모델 다루기와 임베딩

오픈 모델을 직접 띄운다

- 모델 카드 · 라이선스 · 메모리 계산
- 양자화로 작은 GPU에 올리기
- Ollama로 OpenAI 호환 서버 (실습 05)

프롬프트와 임베딩

- 프롬프트 설계 · 구조화 출력 심화 · 전처리
- 글과 이미지를 벡터로, 의미로 찾기
- 실습 09 · 10 (EmbeddingGemma 2)

실습 준비: 코랩에서 **T4 GPU 런타임**이 잡히는지 미리 확인해 둔다